DEEP REINFORCEMENT LEARNING
365.382 (SS26)

Data-Centric RL: Imitation Learning and Offline RL

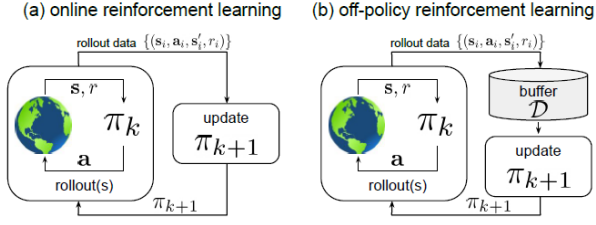
Sohvi Luukkonen
LIT AI LAB
Institute for Machine Learning

JOHANNES KEPLER
UNIVERSITY LINZ

Motivation

Recap: What We Have Done So Far

- So far, we have mostly studied online RL
- The agent repeatedly:
  1. collects data,
  2. updates the policy,
  3. collects more data with the new policy.



- Even with replay buffers, we still collect new environment data.

Levine et al., 2020

The RL Data Problem

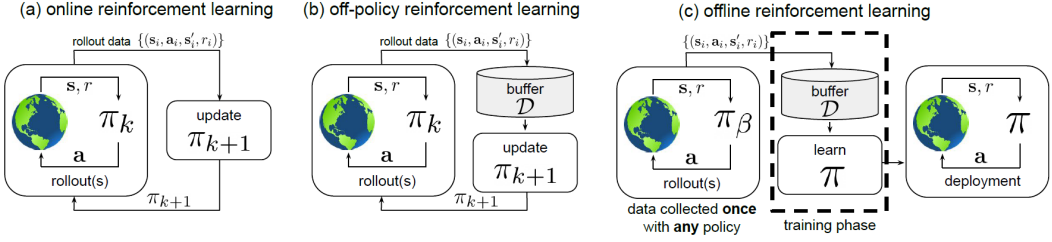
In many real-world settings online data collection is impractical:

- Healthcare: cannot test untrained policies on patients
- Autonomous driving: unsafe exploration in live traffic
- Robotics: physical wear, slow data collection
- Dialogue: expensive to interact with live users at scale




Key question: can we learn good policies from data that already exists?

Three Learning Paradigms

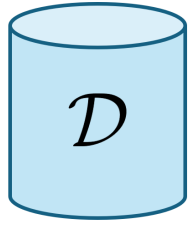


No environment interaction
(also called batch RL)

Levine et al., 2020

Where Does Offline Data Come From?

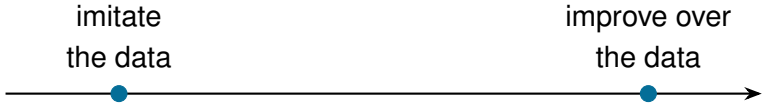
- **Expert demonstrations**
  - Skilled human or existing controller
- **Logs from past policies**
  - Previously deployed systems
- **Human interaction data**
  - Clicks, trajectories, preferences
- **Mixed-quality data**
  - Expert, suboptimal, random, exploratory



We cannot query the environment for more data. Everything must come from D.

Offline RL can only learn behaviors sufficiently represented in the dataset.

Two Ways to Learn from Offline Data



Imitation Learning

- No reward required
- Learn to match demonstrations
- Goal: reproduce expert behavior

Offline RL

- Reward available
- Learn to improve over the data
- Goal: maximize return

Same core challenge:
generalize from the dataset distribution to the deployed policy.

Imitation Learning

Imitation Learning: The Simplest Fixed-Data Setting

- We start with the easiest offline policy learning problem:

  Expert actions, but no rewards

- Demonstration dataset

$$\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{N} \quad a_i \sim \breve{\pi}(\cdot \mid s_i)$$

- Goal

$$\pi_\theta(a \mid s) \approx \breve{\pi}(a \mid s)$$

- Why use IL instead of RL from scratch?
  - ☐ No reward function required
  - ☐ Expert knowledge shortcuts long exploration
  - ☐ Powerful initialization for subsequent RL
- First method

  Behavioral Cloning = supervised learning on demonstrations

Imitation Learning does not optimize reward. It tries to reproduce the demonstrated behavior.

Behavioral Cloning

Behavioral Cloning (BC): treat imitation as supervised learning

$$\hat{\theta} = \arg\min_{\theta} \; \mathbb{E}_{s \sim p_{\text{data}}} \mathcal{L}(\breve{\pi}(s), \, \pi_\theta(s))$$

Loss function depends on action type:

- Continuous actions — MSE:

$$\mathcal{L}_{\text{BC}} = \frac{1}{N} \sum_{k=1}^{N} \|\pi_\theta(s_k) - a_k\|^2$$

- Discrete actions — cross-entropy:

$$\mathcal{L}_{\text{BC}} = -\frac{1}{N} \sum_{k=1}^{N} \log \pi_\theta(a_k \mid s_k)$$

Bain & Sammut, 1995

Case Study: ALVINN (1989)

Autonomous Land Vehicle In a Neural Network

- One of the earliest successful imitation learning systems
- Input:
  - Front-facing camera
  - Laser range finder
- Output:
  - Steering direction
- Trained using: human driving demonstrations
- Successfully drove on real highways at speeds up to 55 mph (88 km/h)
- Main limitation: distribution shift between training and deployment

Pomerleau, 1989

The Distribution Shift Problem

Behavioral Cloning learns from expert trajectories
but is deployed using its own trajectories

Training

$$s \sim p_{\text{data}}(s)$$

- States come from the expert
- Mostly “good” situations
- Policy learns $\pi_\theta(a \mid s)$

Deployment

$$s \sim p_{\pi_\theta}(s)$$

- Small errors $\Rightarrow$ unseen states
- Unseen states $\Rightarrow$ more errors
- Errors compound over time

Training distribution $\neq$ deployment distribution

$$p_{\text{data}}(s) \neq p_{\pi_\theta}(s)$$

Compounding Errors

Theorem (Ross & Bagnell, 2010)

If the learned policy makes mistakes with probability ϵ under the expert distribution, then:

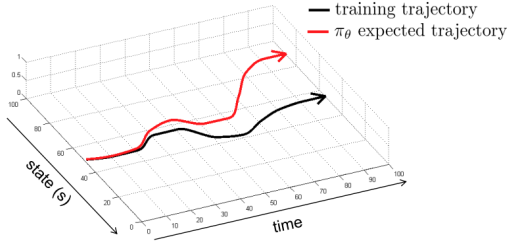
$$\textit{Policy error} \leqslant \textit{Expert error} + T^2\epsilon$$

- Error grows quadratically with horizon T
- Even small per-step mistakes become severe on long trajectories
- Core issue: mistakes change future states

Ross & Bagnell, 2010

Why Does the Error Become Quadratic?

- At every step there is a chance to leave the expert trajectory
- Once off-distribution: future states may also be incorrect
- One early mistake can corrupt: $T - t$ future decisions



T opportunities to fail $\times$ up to T future corrupted states $= O(T^2)$

Case Study: NVIDIA End-to-End Driving (2016)

- CNN trained via BC on human driving footage
- Three cameras: left, center, right
- Off-center cameras: add corrective steering labels
- Effectively augments distribution to include recovery states
- Partial mitigation of distribution shift

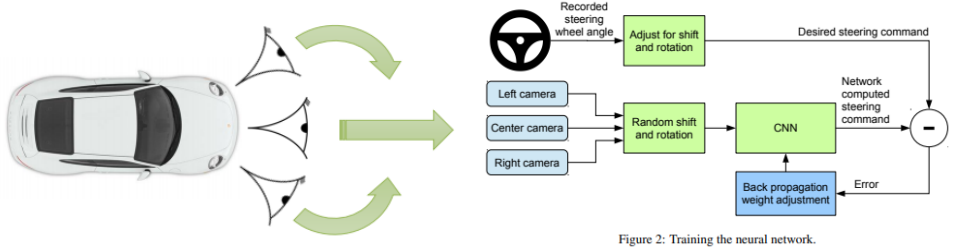


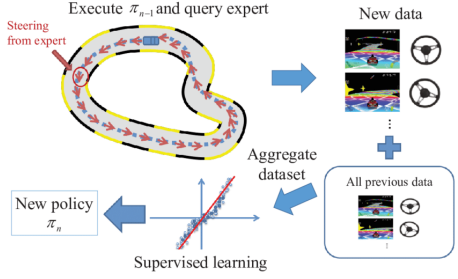
Figure 2: Training the neural network.

Bojarski et al., 2016

DAgger: Dataset Aggregation

Train on the states visited by the learned policy

- BC trains only on expert states
- DAgger rolls out the current learner
- Expert labels those learner-visited states
- Dataset grows over iterations



Result: error growth improves from $O(T^2)$ to $O(T)$

Ross et al., 2011

DAgger Algorithm

Key idea: iteratively train on states visited by the learner

Require: Expert policy $\breve{\pi}$, mixing schedule β_i

1: Initialize dataset D with expert demonstrations
2: Train initial policy π_{θ_1} on D
3: **for** $i = 1, \ldots, N$ **do**
4: Define rollout policy: $\pi_i = \begin{cases} \breve{\pi} & \text{with prob. } \beta_i \\ \pi_{\theta_i} & \text{with prob. } 1 - \beta_i \end{cases}$
5: Sample trajectories using π_i
6: Query expert labels: $D_i = \{(s, \breve{\pi}(s)) \mid s \in \text{trajectories from } \pi_i\}$
7: Aggregate: $D \leftarrow D \cup D_i$
8: Train $\pi_{\theta_{i+1}}$ on D
9: **end for**

Early training: mostly expert Later: mostly learner

DAgger: Cost and Limitations

DAgger reduces distribution shift by iteratively querying the expert at on-policy states

Cost: expert must be queryable at training time

- Human expert: labeling is expensive
- Simulator expert: feasible but requires a simulator
- Real-world expert: often infeasible (patient, unsafe environment)

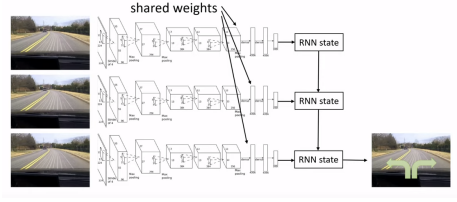
$\Rightarrow$ When expert is not queryable: have to fall back to offline methods

Failure Mode: Non-Markovian Behavior

Problem: humans condition actions on history, not just the current state

$$\breve{\pi}(a_t \mid o_t, o_{t-1}, \ldots, o_0) \neq \breve{\pi}(a_t \mid s_t)$$

- Checking mirrors before lane changes
- Turning after seeing earlier landmarks



Consequence:

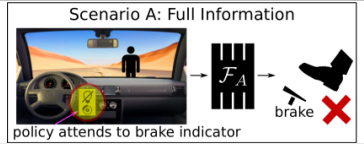
- BC with only s_t as input misses temporal dependencies
- Erratic behavior on states requiring memory

Solutions: recurrent policies, transformer-based policies over observation history

Failure Mode: Causal Confusion

Problem: agent learns spurious correlations rather than true causes

Example: brake light is correlated with braking action



- Expert presses brake ⇒ brake light turns on
- In data: brake action and brake light co-occur
- Agent learns: “brake light ⇒ brake”
- But brake light appears *because* the agent brakes — causality reversed!

Risk: any feature correlated with expert actions can become a spurious cause

- Hard to detect from data alone
- Causal reasoning or disentangled representations may help

---

de Haan et al., 2019

Failure Mode: Multimodal Behavior

Problem: expert demonstrates multiple valid responses to the same state

Example: obstacle can be passed on either side

- Dataset contains a_{left} and a_{right} for similar states
- MSE loss averages the modes:

$$\pi_\theta(s) \approx \frac{a_{\text{left}} + a_{\text{right}}}{2} \quad \Rightarrow \quad \text{crash into obstacle}$$

Solutions:

- Mixture of Gaussians output head (n modes)
- Latent variable models (CVAE, flow-based)
- Autoregressive discretization of continuous actions

Imitation Learning: Summary

Method	Idea	Key Limitation
Behavioral Cloning	SL on expert data	Distribution shift ($T^2\epsilon$)
DAgger	On-policy labeling	Expert access required

Open question: what if we cannot query the expert anymore?

- If rewards are available: $\Rightarrow$ Offline Reinforcement Learning
- If rewards are unavailable: $\Rightarrow$ Inverse Reinforcement Learning

Offline Reinforcement Learning

Offline RL: Problem Statement

Goal: learn a near-optimal policy from a fixed dataset, without new interactions

Setup:

- Fixed dataset: $\mathcal{D} = \{(s_t^i, a_t^i, s_{t+1}^i, r_t^i)\}$
- Dataset collected by behavior policy π_β (may be unknown or mixed)
- $d^{\pi_\beta}(s, a)$ — state-action visitation distribution in $\mathcal{D}$

RL objective is unchanged:

$$\max_\pi \; J(\pi) = \mathbb{E}_{\tau \sim p_\pi(\tau)} \left[\sum_{t=0}^{H} \gamma^t r(s_t, a_t) \right]$$

But we cannot draw new samples from $p_\pi(\tau)$ during training.

Levine et al., 2020

Why Not Just Use Standard Off-Policy RL?

Standard off-policy algorithms (e.g. DQN, SAC) use replay buffers.

Can they just train on a fixed dataset $\mathcal{D}$?

In principle: yes In practice: no

- Replay buffers contain old transitions
- But off-policy RL still collects fresh experience
- New data gradually tracks the current policy
- Offline RL removes this adaptation mechanism entirely

$\mathcal{D} =$ fixed $\Rightarrow$ learned policy π drifts away from π_β

Why Is Offline RL Difficult?

Offline RL only has access to dataset actions

$$(s, a) \sim d^{\pi_\beta}(s, a)$$

But the learned policy may require evaluating:

$$(s, a') \notin \mathcal{D}$$

Dataset contains

- Observed actions
- Observed rewards
- Observed transitions

Dataset does not contain

- Untried actions
- Their rewards
- Their transitions

Offline RL must estimate values beyond the dataset support

Distributional Shift in Q-Learning

Q-learning policy improvement:

$$\pi(s) = \arg\max_a Q(s, a)$$

Problem: $\arg\max_a$ queries $Q(s, a)$ for all actions — including OOD ones Bellman backup:

$$Q(s, a) \leftarrow r(s, a) + \gamma \max_{a'} Q(s', a')$$

- $\max_{a'} Q(s', a')$ evaluates actions not seen in $\mathcal{D}$
- Neural network can extrapolate spuriously: assign high Q to OOD (s', a')
- Error is then propagated back via bootstrapping

Result: greedy policy selects actions with high predicted but low true value

Solution Landscape

Three main families of offline RL methods:

1. Behavior regularization:
   Constrain π to stay close to π_β — limit OOD action selection
2. Conservative value estimation:
   Penalize Q-values for OOD actions during training
3. Sequence modeling:
   Bypass Bellman backups — reframe RL as conditional sequence prediction

Common principle: do not trust the model at out-of-distribution inputs

Behavior Regularization

Idea: prevent the learned policy from choosing unsupported actions

$$\max_{\pi} \; J(\pi) \quad \text{subject to} \quad D(\pi \,\|\, \pi_\beta) \leqslant \delta$$

Common implementations:

- KL divergence penalty: $\mathcal{L} = -J(\pi) + \lambda \, D_{\mathsf{KL}}(\pi \,\|\, \pi_\beta)$
- BC auxiliary loss alongside RL objective: $\mathcal{L} = -J(\pi) + \lambda \, \mathcal{L}_{\mathsf{BC}}$
- Support constraint: $\pi(a|s) = 0$ if $(s, a) \notin \mathcal{D}$

Trade-off:

- Tight constraint $\Rightarrow$ safe but limits improvement (close to BC)
- Loose constraint $\Rightarrow$ may still extrapolate dangerously

Conservative Q-Learning (CQL)

Key idea: explicitly push down Q-values for OOD actions

$$\min_Q \Big(\underbrace{\mathbb{E}_{s\sim\mathcal{D},\, a\sim\mu(a|s)}[Q(s,a)]}_{\text{minimize OOD Q-values}} - \underbrace{\mathbb{E}_{(s,a)\sim\mathcal{D}}[Q(s,a)]}_{\text{maximize in-data Q-values}} + \mathcal{L}_{\text{TD}} \Big)$$

where μ is a broad distribution over actions (e.g. uniform)

- Q_{CQL} is a lower bound on the true Q-function: $Q_{\text{CQL}} \leqslant Q^\pi$
- Policy trained to maximize conservative Q still improves
- Overestimation on OOD actions is suppressed

In practice: strong performance across locomotion, manipulation, and real-world benchmarks

---

Kumar et al., 2020

Implicit Q-Learning (IQL)

Key idea: never query $Q(s, a)$ for actions outside the dataset Standard target:

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a') \qquad \text{(queries OOD } a'\text{)}$$

IQL approach: replace $\max_{a'}$ with expectile regression on $V(s)$

$$V(s) = \arg\min_{V} \; \mathbb{E}_{a \sim \mathcal{D}(s)}[\mathcal{L}^{\tau}(Q(s, a) - V(s))]$$

where $\mathcal{L}^{\tau}(u) = |\tau - \mathbf{1}[u < 0]|\, u^2$ is an asymmetric (expectile) loss.

- Only evaluates dataset actions — no OOD queries.
- $\tau \approx 1$: $V(s)$ approximates $\max_{a \in \mathcal{D}} Q(s, a)$
- Enables offline Q-learning with a subsequent policy extraction step

---

Kostrikov et al., 2021

Expectile Regression Intuition

IQL uses an asymmetric regression loss

$$\mathcal{L}^\tau(u) = |\tau - \mathbf{1}[u < 0]|\, u^2 \quad \text{where} \quad u = Q(s,a) - V(s)$$

Case 1: $Q(s,a) > V(s)$

$$\Rightarrow u > 0$$
$$\Rightarrow \mathbf{1}[u < 0] = 0$$
$$\Rightarrow \mathcal{L}^\tau(u) = \tau u^2$$
$$\tau \approx 1 \Rightarrow \mathcal{L}^\tau(u) \approx u^2$$

high-Q actions receive large penalties

Case 2: $Q(s,a) < V(s)$

$$\Rightarrow u < 0$$
$$\Rightarrow \mathbf{1}[u < 0] = 1$$
$$\Rightarrow \mathcal{L}^\tau(u) = (1-\tau)u^2$$
$$\tau \approx 1 \Rightarrow \mathcal{L}^\tau(u) \approx 0$$

low-Q actions receive small penalties

To minimize $\mathcal{L}^\tau$, $V(s)$ is pushed up towards high-Q actions and down away from low-Q actions $\Rightarrow V(s)$ approximates a soft maximum over dataset actions.

Decision Transformer

Key idea: reframe offline RL as conditional sequence modeling

Input: trajectory of (return-to-go, state, action) triples

$$\hat{R}_t = \sum_{t'=t}^{H} r_{t'}, \qquad \text{sequence: } \left(\hat{R}_1, s_1, a_1,\ \hat{R}_2, s_2, a_2,\ \ldots\right)$$

- Train GPT-style causal transformer to predict actions given return-to-go
- At test time: condition on desired return $\hat{R}_1 = R_{\text{target}}$
- Model autoregressively generates actions that achieve that return

Advantages: no Bellman backups, no OOD extrapolation, scales with data and model size

Limitation: Less effective at trajectory stitching than value-based methods

Chen et al., 2021

Offline RL: Method Comparison

Method	Family	OOD	Stitching
BC regularization	Behavior reg.	Avoided	Limited
CQL	Conservative Q	Penalized	Yes
IQL	Conservative Q	Avoided	Yes
Decision Transformer	Seq. modeling	Avoided	No

Stitching: combining sub-optimal data to form a better-than-dataset policy

$\Rightarrow$ Methods based on Bellman backups (CQL, IQL) can stitch; sequence models cannot

Summary

IL and Offline RL: Side by Side

Aspect	Imitation Learning	Offline RL
Objective	Match expert	Maximize reward
Data quality	Expert demos	Any π_β
Reward required	No	Yes
Core challenge	Distribution shift	Q-value extrapolation
Key algorithm	DAgger	CQL, IQL
Upper bound	Expert level	Above dataset possible

Common Thread: Distributional Shift

Imitation Learning:

- Training on $p_{\text{data}}(s)$, deployment on $p_{\pi_\theta}(s)$
- Compounding errors: $J(\pi_\theta) \leqslant J(\breve{\pi}) + T^2\epsilon$
- Fix: collect on-policy labels (DAgger) $\Rightarrow T\epsilon$

Offline RL:

- Q-function evaluated at OOD (s, a) pairs
- Overestimation propagates via Bellman backups
- Fix: constrain policy (behavior reg.), penalize OOD Q (CQL), avoid OOD queries (IQL)

Central lesson: learning from a fixed distribution, while generalizing to a new one, requires explicit design choices to control the resulting shift.

Open Problems

- Data quality and coverage: how much does dataset composition matter?
- Offline evaluation: how to assess offline RL without environment interaction?
- Offline-to-online transfer: can we fine-tune offline policies with limited interaction?
- Reward labeling: combining offline RL with reward learning and IRL
- Foundation models: can large pretrained models provide priors for offline RL?